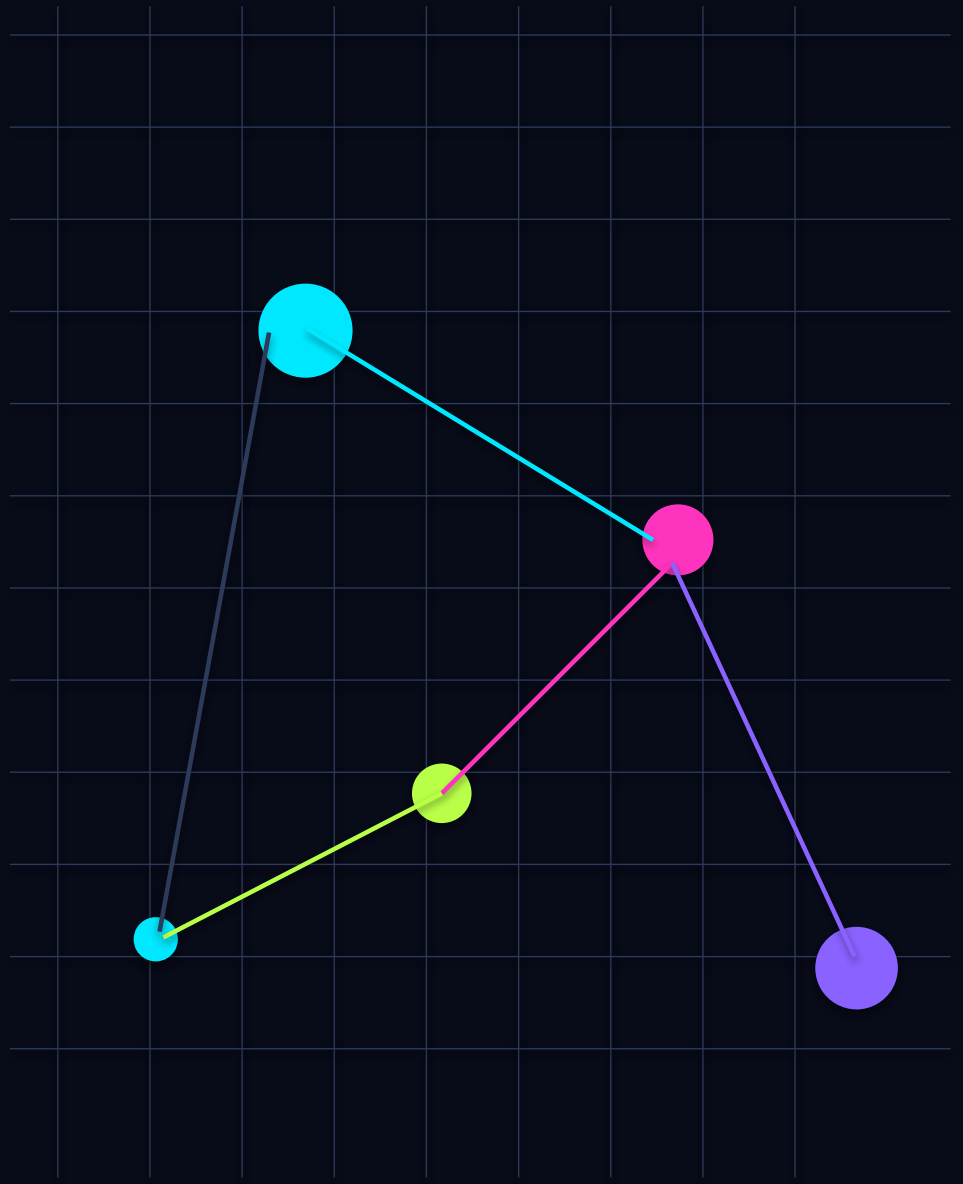
FIELD GUIDE / 2026

How to integrate AI into the daily QA process

From requirements to reliable releases — with humans in control.



MANUAL QA + AUTOMATION QA

Context-aware assistance. Repeatable routines. Traceable outcomes.

AI should remove friction — not ownership

Treat AI as a QA co-pilot embedded in the workflow, not a separate destination.



CORE RULE: AI proposes and executes bounded tasks; QA owns risk, acceptance, and release confidence.

Turn project context into test-ready knowledge

Use source material as inputs; keep the generated artifacts linked to the requirements.

01 Ingest context

Requirements, docs, Figma/designs, API schemas, DB schemas, existing tests and known defects.

02 Analyze

Identify actors, state transitions, edge cases, data constraints, dependencies and ambiguity.

03 Generate

Context-based test cases, risk-based coverage, exploratory charters and test data ideas.

04 Publish

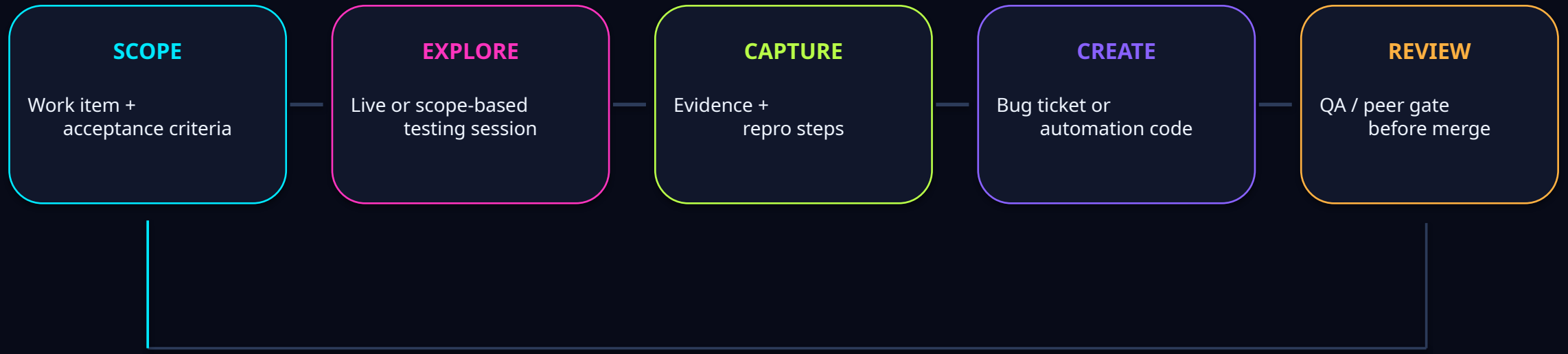
Test plans, checklists, traceability, test summaries and reusable documentation.

HUMAN CHECKPOINTS

- Confirm business intent
- Challenge missing cases
- Validate testability
- Approve before publishing

One loop for manual and automation QA

The same context can drive exploration, defect reporting, code generation and review.



Feedback loop: outcomes update context, coverage and the next test session.

From intent to a reviewed pull request

Codegen is a starting point. Reliability comes from assertions, maintainability and review.

01 **Choose a scenario**

02 **Generate a first draft**

03 **Add assertions & data setup**

04 **Run targeted checks**

05 **Open PR with evidence**

PR QUALITY GATE

- ✓ Assertions validate behavior, not just presence
- ✓ No brittle selectors or hidden waits
- ✓ Fixtures, data and cleanup are understood
- ✓ Diff is focused; no unrelated refactors
- ✓ CI results and screenshots/logs are attached

Outcome: faster first draft + consistent review — not unreviewed AI-generated test code.

Move repeatable maintenance into scheduled routines

Automate the boring checks; keep changes observable, reversible and bounded.

ON DEMAND

Code hygiene skill

- Optimize confusing or duplicated code
- Remove provably dead code
- Explain the diff and risk
- Run focused tests before proposing changes

Output: small, reviewable patch

SCHEDULED

Skipped-test reconciliation

- Read test ↔ work-item mapping
- Inspect linked work-item statuses
- Skip only when the rule matches
- Unskip when work is active / complete per policy

Output: status diff + audit trail

Safety rails: dry-run first • allowlist scope • log every decision • never silently delete tests

Keep responsibilities explicit

AI can accelerate execution; accountability stays with the people who understand the product.

WORK ITEM	MANUAL QA	BOTH QA ROLES	HUMAN GATE
Context → test cases	Primary	Reuse context	Review coverage
Test documentation	Generate / update	Contribute evidence	Approve source of truth
Exploration + defects	Participate	Run + file tickets	Confirm severity / impact
Codegen + PRs	—	Draft, test, propose	Review diff + CI
Code hygiene routine	—	Analyze / propose patch	Approve destructive changes
Skipped-test routine	—	Reconcile by policy	Own mapping + exceptions

Adopt in layers, prove value at each step

Start with low-risk assistance, then graduate to bounded actions and scheduled routines.

PHASE 1

ASSIST

Generate test cases & docs
Human review only

PHASE 2

CONNECT

Create defects, link evidence
Use existing tracker APIs

PHASE 3

PROPOSE

Generate tests + open PRs
CI and reviewer required

PHASE 4

SCHEDULE

Hygiene + skip reconciliation
Dry-run → audit → controlled write

Measure: review time • escaped defects • flaky-test rate • PR cycle time • skipped-test accuracy

THE TAKEAWAY

AI becomes useful when it joins the workflow.

Not another chatbot tab. A connected loop from context to evidence to action.

CONTEXT FIRST

Ground outputs in real project artifacts.

HUMAN GATES

Review risk, semantics and destructive changes.

AUDITABLE ROUTINES

Log decisions; make actions reversible.

The goal: more meaningful QA work, less repetitive coordination.